WAVE User Manual

Leandro Cavalcanti de Almeida
(leandro.almeida@ifpb.edu.br)
Paulo Ditarso Maciel Jr.
(paulo.maciel@ifpb.edu.br)
Danilo Cavalcante Beuttenmuller
(danilo.cavalcante@academico.ifpb.edu.br)
Matheus Faelson de Almeida Valério
(matheus.faelson@academico.ifpb.edu.br)
Caio Luiz Lacerda Terto Silva
(caio.luiz@academico.ifpb.edu.br)
Icaro Machado da Silva
(icaro.silva@academico.ifpb.edu.br)
Ivalcleb Benigno de Souza
(ivalcleb.leoncio@academico.ifpb.edu.br)

https://github.com/1valcl3b/last_wave
version 1.2, March 18, 2026

Abstract

This manual presents the WAVE tool, a workload generator for verifiable and application-agnostic experiments, used in testing for video-on-demand and key-value storage. WAVE can generate workload patterns such as *sinusoidal*, *flashcrowd*, and *step*, with support for network micro-bursts. In this work, WAVE is integrated with the Mininet emulator, providing researchers with greater control over network configuration, including parameters such as delay and packet loss. Currently, the *linear* and *tree* topologies are supported.

Contents

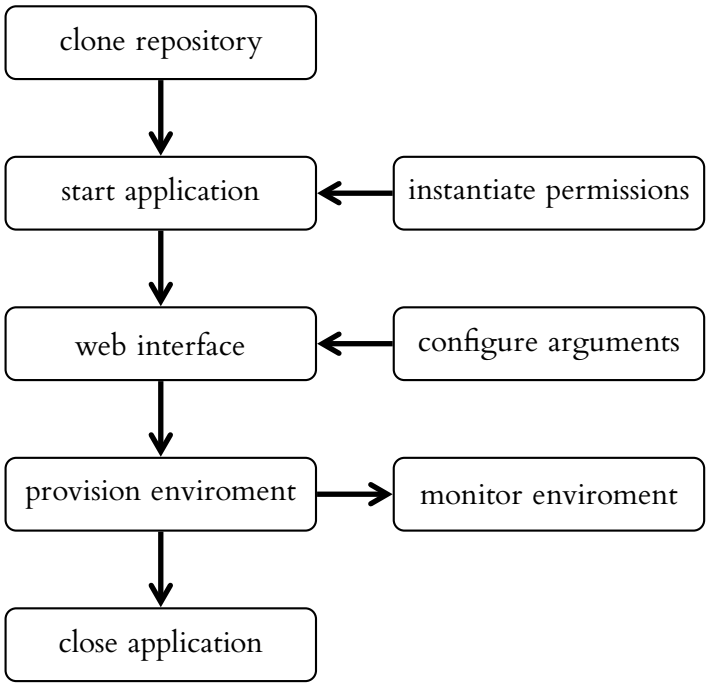
1 Checking the Required Requirements 2

2 Downloading the Code and Starting the Environment 3

3 Interacting with WAVE WEB 9

4 Ending the WAVE Execution 12

Quick Workflow Guide



1 Checking the Required Requirements

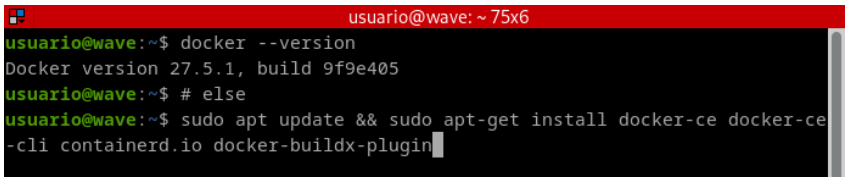
1.1 Checking if **`Python3`** [8] is installed and it's version:

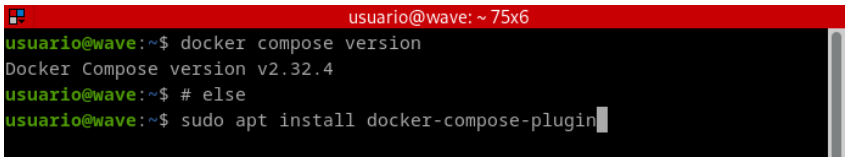
```
usuario@wave: ~ 75x7
usuario@wave:~$ python3 --version
Python 3.11.2
usuario@wave:~$ # else
usuario@wave:~$ sudo apt update && sudo apt install python3
```

1.2 Additionally, the **`VirtualEnv`** virtual environment is required :

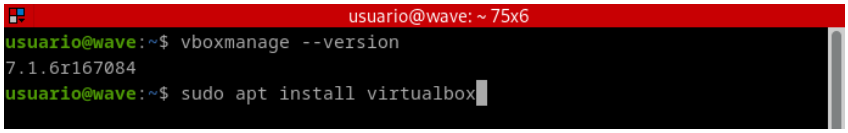
```
usuario@wave: ~ 75x8
usuario@wave:~$ sudo apt list | grep python3-venv

WARNING: apt does not have a stable CLI interface. Use with caution in scri
pts.

python3-venv/stable,now 3.11.2-1+b1 amd64 [installed]
usuario@wave:~$ # else
usuario@wave:~$ sudo apt update && sudo apt install python3-venv
```

1.3 Checking the **`Docker`** [3] and **`docker compose`** components:

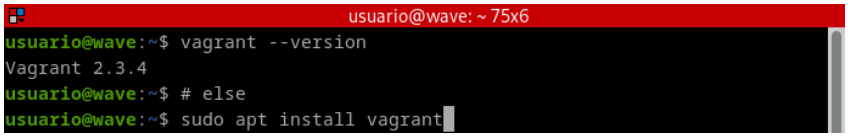




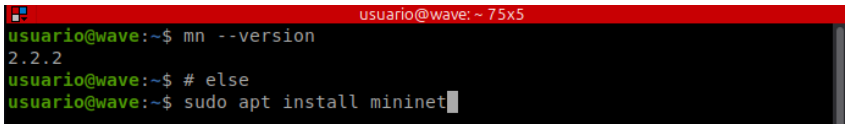
1.4 Checking what version of **`VirtualBox`** [11] is installed:



1.5 Checking what version of **`Vagrant`** [10] is installed:



1.6 Checking what version of **`Mininet`** [6] is installed:



We recommend installing Mininet from the official website, as it provides the most up-to-date version: https://mininet.org/download/ Although Mininet can also be installed using apt install mininet, the version available in the distribution repositories may not be the most recent one.

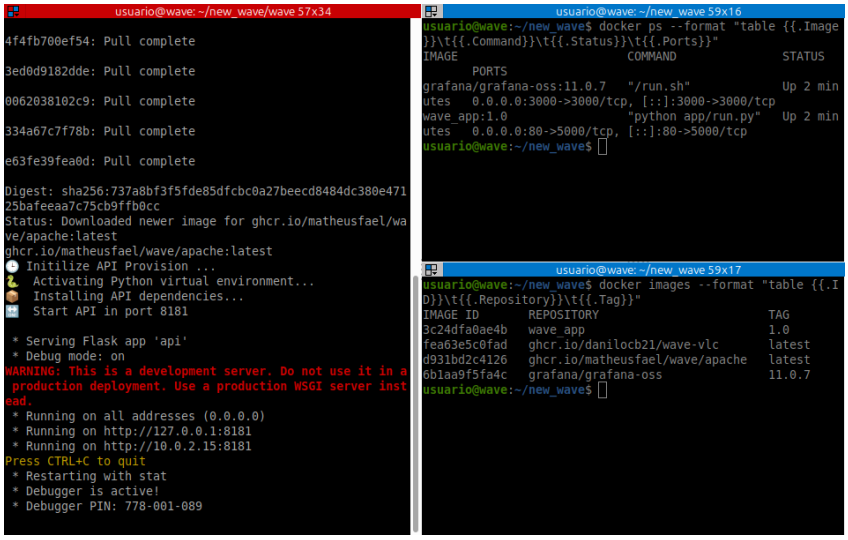
The versions shown in the figures for steps 1.1–1.5 were those tested at the time of this manual's creation.

2 Downloading the Code and Starting the Environment

2.1 Cloning the official repository and starting the system:

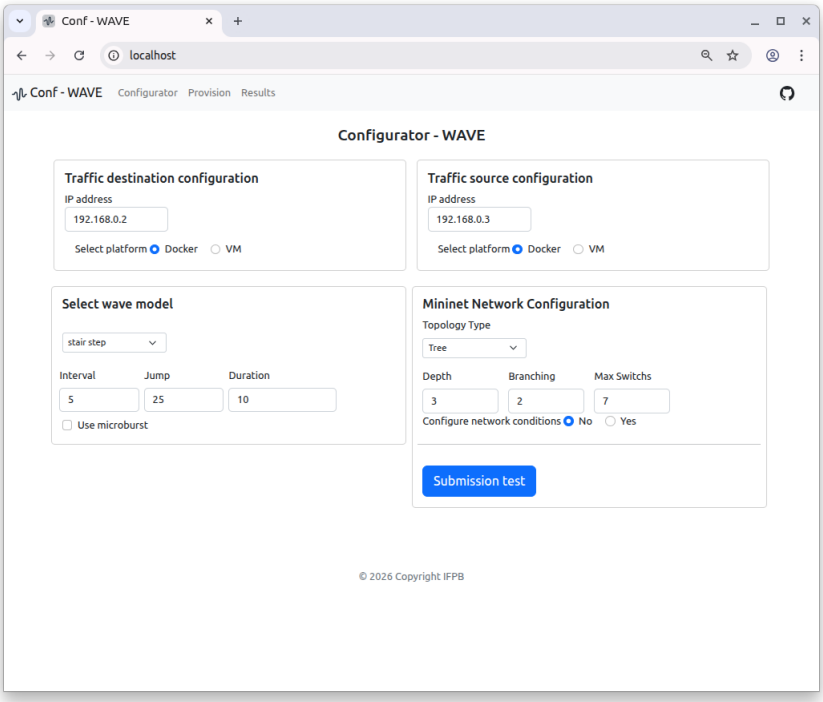
```
$ git clone https://github.com/1valcl3b/last_wave.git
$ cd last_wave/wave
$ ./app-compose.sh --start
```

2.2 Checking the execution in a **Docker** environment:



As can be seen in the figure above, the WAVE Initialization module uses two containers for its execution: `wave_app` and `grafana-oss`. On the left side of the figure, we have the output of the WAVE startup command, which corresponds to the output of line 3 of the code shown in section 2.1.

2.3 The WAVE Web module can be accessed via a browser



The form contains fields for entering network data for both the traffic load source and destination. In addition to specifying the IP address, the user can choose how the environment will be provisioned, either through a container or a virtual machine, with configurable memory

size and number of virtual CPUs. It is also possible to configure the network topology through user-defined parameters. Currently, the WAVE supports linear and tree topologies. Finally, the user can select which workload model to apply, such as sinusoid, flashcrowd, or step, and optionally enable the use of micro-burst traffic.

2.4 Virtualization environment started by the Provisioning module:

The virtual machines used as the source and destination of the traffic load are configured from the `app/provision` directory. For this purpose, the *Vagrant* tool and the *VirtualBox* hypervisor are instantiated to run the environment. This directory contains a pre-edited `Vagrantfile`, whose configurations are provided by another file called `config.yaml`. The latter is generated after filling out the form in the WAVE WEB module, containing the arguments assigned by the user. Additionally, two pre-configured `vagrant boxes` (`wave-server` and `wave-client`) have been set up with the necessary software for generating load between the source and destination (`IPerf` [5] and traffic load models), respectively referred to as Client VM and Server VM. If the `boxes` are not yet installed on the local machine, they will be downloaded during the system's first execution. Below is the `Vagrantfile` code responsible for provisioning the environment.

```ruby
# -*- mode: ruby -*-
# vi: set ft=ruby :

Vagrant.require_version '>= 1.6.0'
VAGRANTFILE_API_VERSION = '2'

require 'yaml'


machines = YAML.load_file(File.join(File.dirname(__FILE__), 'config.yaml'))

server_cfg = machines.find { |m| m['traffic'] == 'server' }
client_cfg = machines.find { |m| m['traffic'] == 'client' }


ls= File.read("/tmp/last_switch.txt").strip


$SCRIPT = <<-EOF
server_ip=#{server_cfg['ip']}
echo -e "$server_ip server" | sudo tee -a /etc/hosts
client_ip=#{client_cfg['ip']}
echo -e "$client_ip client" | sudo tee -a /etc/hosts
```

```
EOF

Vagrant.configure(VAGRANTFILE_API_VERSION) do |config|


  machines.each do |entrada|

    next unless entrada['traffic']

    config.vm.define entrada['traffic'] do |set|

      set.vm.hostname = entrada['traffic']

      if entrada['traffic'] == "server"
        set.vm.box = "ifpb/wave-server"
        set.vm.hostname = "wave-server"
      else
        set.vm.box = "wave/client"
        set.vm.hostname = "wave-client"

        set.vm.synced_folder "./logs", "/home/vagrant/wave/logs"
      end

      if entrada['traffic'] == "client"
        set.vm.network "public_network", ip: entrada['ip'], bridge: "s1"
        # set.vm.network "private_network", ip: "192.168.56.10"

      elsif entrada['traffic'] == "server"
        set.vm.network "public_network", ip: entrada['ip'], bridge: ls
        set.vm.network "private_network", ip: "192.168.56.50"
      end

      set.vm.provider "virtualbox" do |vb|
        vb.memory = entrada['ram']
        vb.cpus = entrada['vcpu']
      end
      set.vm.provision "shell", inline: $SCRIPT
    end
  end
end
```

In the context of the WAVE project, Docker is employed as a portable,

lightweight, and scalable solution for containerized deployment of the application's core services. This approach significantly streamlines the environment setup, as Docker containers are utilized to encapsulate both the client and server components in isolated environments. The use of Docker further facilitates the rapid replication of experimental setups across different systems, ensuring a high degree of consistency and minimizing the time required for environment configuration.

For the provisioning of the environment, Docker Compose is employed to launch the containers automatically as soon as the user initiates the provisioning process. Subsequently, the client container, which runs VLC Media Player (VLC), remains in a waiting state, ready to receive commands through the web interface. The server container, in turn, is configured with Apache HTTP Server (Apache) to host the video content. Docker Compose serves as an orchestration tool, enabling the definition and management of multi-container applications. The various services that comprise WAVE—specifically the client and server components—are articulated and managed within isolated containers. This approach not only simplifies the orchestration of these services but also ensures the rapid deployment and scalability of the environment. Below is the docker-compose.yaml file responsible for provisioning the environment.

```yaml
services:
  apache:
    image: ghcr.io/ifpb/new_wave/wave-apache
    container_name: server
    #ports:
    #- "80:80"

  client_container:
    image: ghcr.io/ifpb/new_wave/wave-vlc
    container_name: client
    privileged: true
    #environment:
      #- DISPLAY=${DISPLAY}
    volumes:
      - /etc/localtime:/etc/localtime:ro
      - ./logs:/home/vlc/logs
      - /tmp/.X11-unix:/tmp/.X11-unix
    depends_on:
      - apache
```

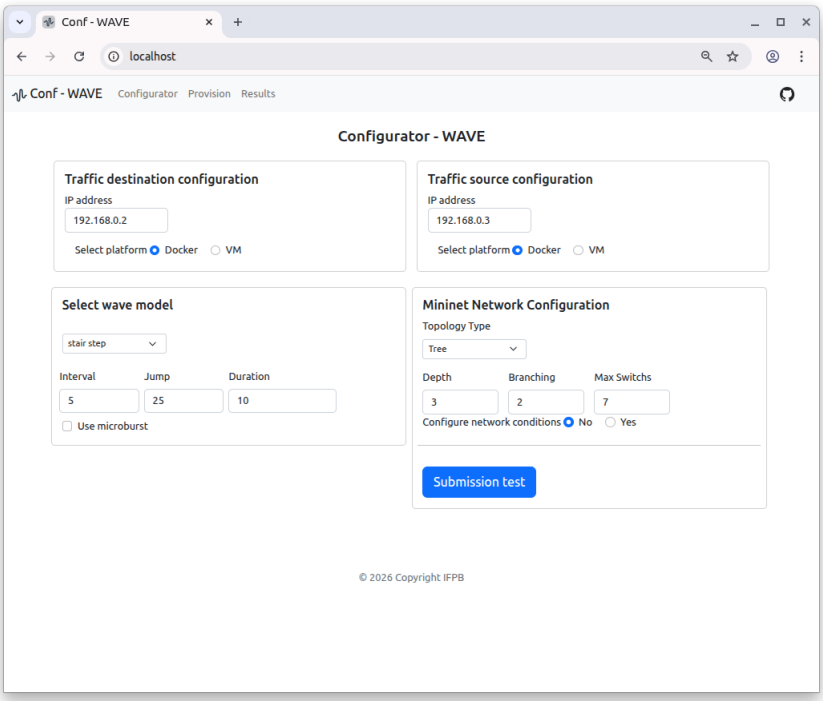
In WAVE, Mininet is used to enable the creation of configurable network topologies during experiments. To this end, topologies

are generated through Python scripts that utilize the Mininet API. This approach ensures greater flexibility, allowing the configuration of certain network characteristics, such as delay and packet loss, in addition to automating the process of setting up the experimental environment. During the experiment configuration, the parameters defined by the user in the WAVE Web interface are stored in a file called *config.yaml*. This file is later read by the scripts responsible for network provisioning. Depending on the selected topology, WAVE automatically executes the corresponding script: *net-tree.py* for *Tree* topologies and *net-linear.py* for *Linear* topologies. These scripts, in turn, use the Mininet API to instantiate switches and create links according to the defined parameters, automatically building the network topology as specified by the user. Below is an excerpt from *net-linear.py*.

```python
def montar_linear(net, num_switches, delay,loss):
    switches = []

    for i in range(num_switches):
        nome = f"s{i + 1}"

        sw = net.addSwitch(nome, cls=OVSSwitch,
        ↪  failMode='standalone')
        switches.append(sw)

        if i > 0:

            link_parametros={}

            if delay:
                link_parametros['delay']=delay

            if loss:
                link_parametros['loss'] = float(loss)

            net.addLink(switches[i -1], sw, **link_parametros)


    return switches
```

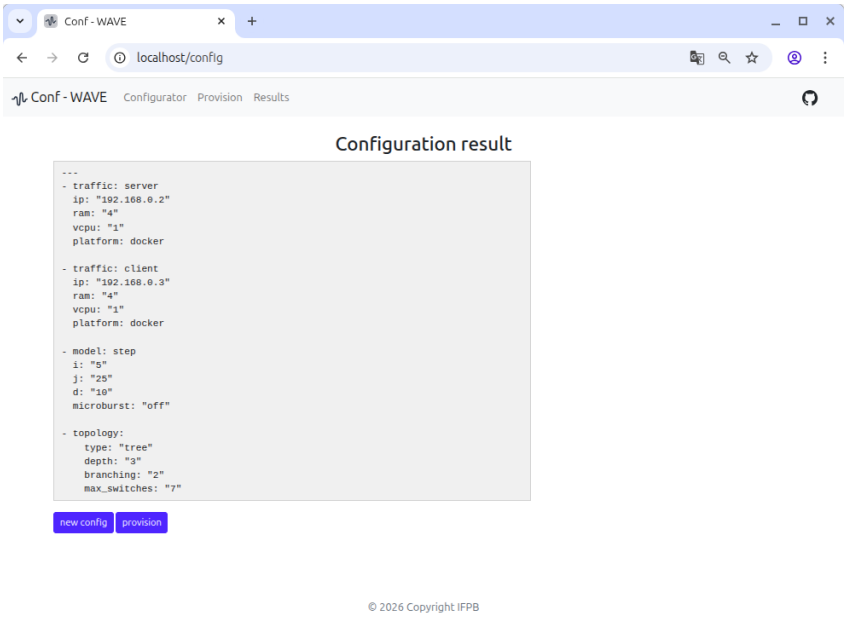
3 Interacting with WAVE WEB

3.1 Once the system is initialized, the user must enter the arguments:

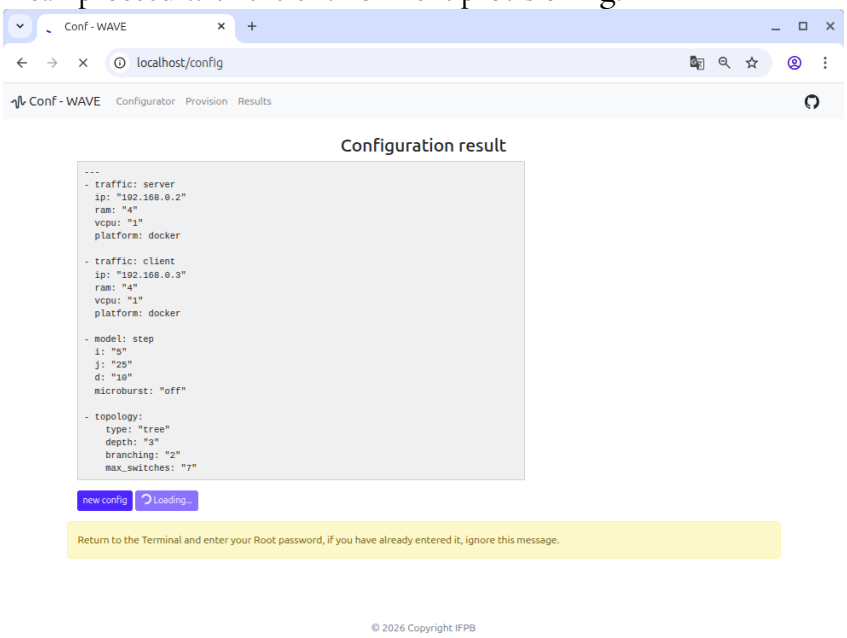


The previous figure illustrates the completion of the web form with the necessary parameters for executing a workload based on a *stair step* model. First, the IP addresses of the Server Container and the Client Container are configured. It is important to pay attention to the IP address configuration: if the user defines IPs belonging to the network range of a virtualization tool (for example, the `192.168.56.0/24` range of VirtualBox), the traffic will be directed to that network, instead of being forwarded to the Mininet network. At the bottom of the form, a *stair step* load model is instantiated through its respective arguments. For a better understanding of the supported load models, reading [1, 2, 9] is recommended.

3.2 Consolidation of inserted arguments:



After entering the arguments in the homepage form, WAVE WEB displays a confirmation page for the input data. These arguments are edited in a YAML-formatted file called `config.yaml`. If the user notices any errors in the entered arguments, they can return to the homepage using the new configuration button or the menu in the top bar of the web page. Once the data has been verified, the user can proceed with the environment provisioning.

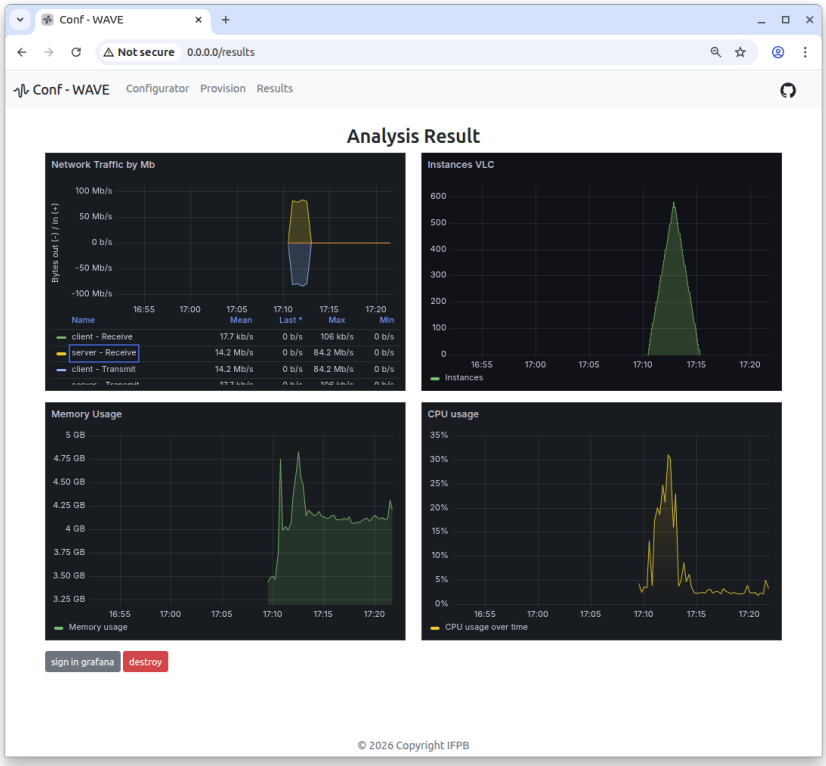


When Docker is used, the provisioning process operates similarly to the virtual machine-based setup. In this scenario, the Server container is launched with Apache pre-installed and configured to serve video content over HTTP. Apache is responsible for providing the media files to the Client container, which is instantiated with

`VLC` (VideoLAN Client) pre-installed. The `VLC` runs in client mode, being configured to request and stream the video content served by Apache. Upon initialization, one of the video workload models is automatically selected based on user input, determining the traffic behavior—such as varying the number of concurrent video requests or simulating sudden bursts in demand.

Alternatively, in the virtual machine-based scenario, the Provisioning module automatically configures and starts the two VMs responsible for generating and receiving the traffic load. In this context, the Server VM is launched with `IPerf` running in server mode, ready to receive traffic on its default port (5201). The Client VM, in turn, is instantiated with `IPerf` pre-installed, along with the `Python` code responsible for the *sinusoid*, *flashcrowd*, and *step* workload models. One of these models is automatically selected based on user-defined parameters, and the corresponding workload is sent to the Server VM. Depending on the selected model, `IPerf` instances in client mode are dynamically started and terminated to emulate the desired behavior. The number of running instances can be verified through the recorded *log* files on the Client VM. Additionally, the Monitoring module allows users to track various execution metrics of the Server VM.

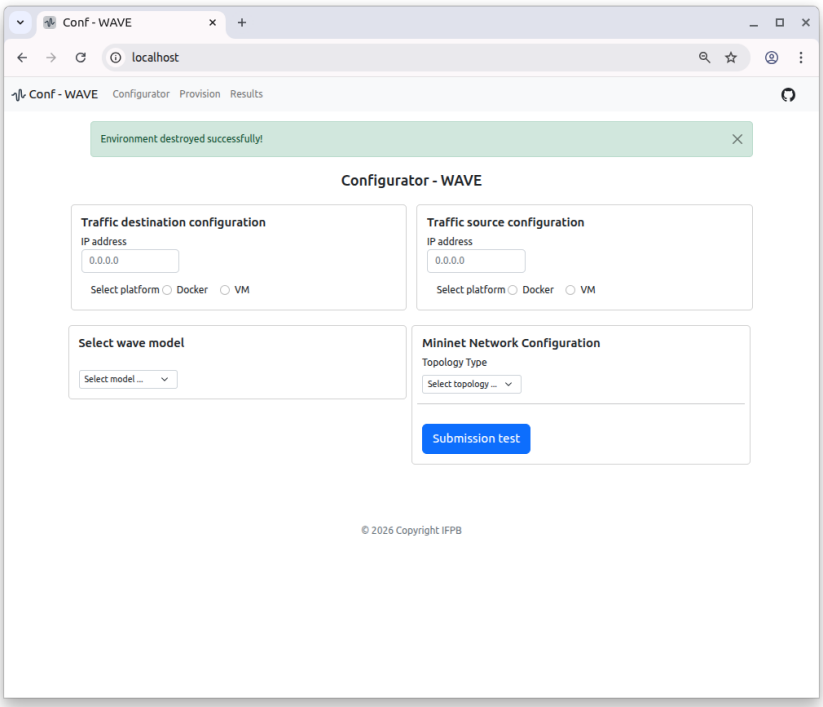
3.3 Once the environment is provisioned, a results page follows:



The Monitoring module is responsible for collecting and displaying execution metrics from the Server VM. For this purpose, the

server's `vagrant box` is preconfigured with the `Prometheus` [7] and `node_exporter` tools. Using `Prometheus` as a data source, the `Grafana` container can display all metrics collected by `node_exporter`, including various indicators related to CPU, memory, and disk usage, as well as network traffic transmission and reception. After VM provisioning, the page above displays the rate of bytes received and sent through the Server VM's network interfaces. This enables, for example, real-time monitoring of the experiment, which involves sending traffic between the source and destination according to a specific load model. Futhermore, it's also possible to see the data used by the container via the cadVisor container that starts with the web app. Additionally, users can open a new web page with the `Grafana` interface[1] [4] to explore other relevant metrics or even configure new custom dashboards.

3.4 At any moment, it's possible to close the containers/virtual machines:



By clicking the `destroy` button on the traffic verification page, the user is redirected to the homepage, where they can restart the experiment if desired.

4 Ending the WAVE Execution

4.1 Finalizing and removing the container environment:

[1] For the first login, the default username and password are `admin`.

```
$ ./app-compose.sh --destroy
```

By running the command above, the user terminates the WAVE WEB module and removes the containers responsible for the other initiated modules. To restart the entire system, simply execute the same command, replacing the `-destroy` argument with `-start`, as indicated in line 3 of the code in section 2.1.

References

[1] Leandro Almeida, Fábio Verdi, and Rafael Pasquini. Estimando métricas de serviço através de in-band network telemetry. In *Anais do XXXIX Simpósio Brasileiro de Redes de Computadores e Sistemas Distribuídos*, pages 252–265, Porto Alegre, RS, Brasil, 2021. SBC.

[2] Ismail Ari, Bo Hong, Ethan Miller, Scott Brandt, and Darrell Long. Managing flash crowds on the internet. In *MASCOTS 2003*, pages 246– 249, 11 2003.

[3] Docker: Accelerated, Containerized Application Development. `https://www.docker.com/`.

[4] Grafana: The open observability platform | Grafana Labs. `https://grafana.com/`.

[5] IPerf - the ultimate speed test tool for TCP, UDP and SCTPTEST the limits of your network + internet neutrality test. `https://iperf.fr/`.

[6] Mininet - An Instant Virtual Network on your Laptop (or other PC). `https://mininet.org/`.

[7] Prometheus - Monitoring system & time series database. `https://prometheus.io/`.

[8] Welcome to Python.org. `https://www.python.org/`.

[9] Rolf Stadler, Rafael Pasquini, and Viktoria Fodor. Learning from network device statistics. *J. Netw. Syst. Manag.*, 25(4):672–698, 2017.

[10] Vagrant by HashiCorp. `https://www.vagrantup.com/`.

[11] Oracle VM VirtualBox. `https://www.virtualbox.org/`.